

PEFT, LoRA, QLoRA

Chapitre 3 — Adapter les LLMs avec peu de ressources

Pourquoi ne pas faire de full fine-tuning ?

Une réponse calculatoire d'abord

- CamemBERT-base : 110M paramètres → 440 Mo en float32, et ~1,8 Go avec l'optimiseur Adam (params + gradients + 2 moments)
- Avec activations : 8 à 12 Go — hors de portée d'un GPU T4 (16 Go) avec un batch size raisonnable
- LLaMA-7B : 28 Go en float32 — le full fine-tuning est impossible sur du matériel académique standard

Réponse plus profonde : les LLMs pré-entraînés ont déjà une représentation riche du langage. Adapter à une tâche spécifique ne requiert qu'une petite fraction stratégique des paramètres — c'est le fondement de PEFT.

Plan du chapitre

- 1. Le paysage du fine-tuning : un spectre de compromis
- 2. LoRA : décomposition de rang faible
- 3. Hyperparamètres LoRA : ce que chaque curseur fait
- 4. QLoRA : quantisation et LoRA combinés
- 5. Les autres méthodes PEFT
- 6. Entraînement LoRA en pratique : mT5 et CamemBERT
- 7. Scheduler cosine, warmup et arrêt prématuré
- 8. Connexion aux décisions du module

PARTIE 1

Le paysage du fine-tuning : un spectre de compromis

Full fine-tuning : le cas de référence

Tous les paramètres sont mis à jour

$$\text{Mémoire_Adam} = N \times (4+4+4+4) \text{ octets} = 16N \text{ octets}$$

paramètres + gradients + moment 1 (m_t) + moment 2 (v_t) — chacun en float32 (4 octets)

- Méthode la plus expressive : tous les poids peuvent s'adapter
- La plus coûteuse en mémoire, en calcul, et en risque de sur-apprentissage sur petit corpus
- CamemBERT-base (N=110M) : $16 \times 110 \times 10^6 \approx 1,76$ Go pour l'optimiseur seul

Taxonomie des méthodes PEFT

Geler la majorité des poids, n'entraîner qu'une fraction

Ajout de paramètres

Adapters, LoRA, prefix tuning : des modules supplémentaires sont insérés dans l'architecture.

Sélection de paramètres

BitFit : un sous-ensemble des poids existants (les biais, ~0,1%) est mis à jour.

Reparamétrisation

LoRA (aussi) : les paramètres entraînables sont exprimés dans un espace de dimension réduite.

Prompting appris

Prompt tuning : le prompt d'entrée est complété par des vecteurs continus entraînables.

LoRA appartient à deux catégories (ajout + reparamétrisation) — une méthode hybride.

Comparatif des approches PEFT

Méthode	Params entraînés	Mémoire	Inférence modifiée	Usage recommandé
Full fine-tuning	100 %	Très élevée	Non	Corpus large, GPU puissant
LoRA	0,1–1 %	Faible	Non (fusion possible)	Cas général, GPU limité
QLoRA	0,1–1 %	Très faible	Non	Très grand modèle, GPU limité
Adapters	~3 %	Modérée	Oui (latence +)	Tâches multiples, même modèle
Prefix tuning	0,1 %	Très faible	Oui (séquence +)	Tâches de génération
Prompt tuning	< 0,01 %	Minimale	Oui (token +)	Modèles très grands
BitFit	~0,1 %	Minimale	Non	Classification, BERT-like

PARTIE 2

LoRA : décomposition de rang faible

L'hypothèse fondamentale

Hu et al., 2022 — un rang intrinsèque faible

- Les mises à jour ΔW nécessaires pour adapter un modèle pré-entraîné vivent dans un sous-espace de faible dimension
- S'appuie sur Li et al. (2018) : le paysage de perte d'un réseau sur-paramétrisé a une dimension intrinsèque bien inférieure au nombre total de paramètres

$$\Delta W = B A \quad \text{avec } B \in \mathbb{R}^{(d \times r)}, A \in \mathbb{R}^{(r \times k)}, r \ll \min(d, k)$$

W_0 reste gelé ($\nabla W_0 L = 0$) ; seules A et B reçoivent des gradients.

Le forward pass adapté

$$h = W_0 x + \Delta W x = W_0 x + B A x$$

- W_0 : poids pré-entraînés, gelés
- B et A : matrices LoRA entraînaibles, de petite taille
- La sortie est la somme du chemin gelé et du chemin adapté

Initialisation et mise à l'échelle

Le début de l'entraînement doit être neutre

- A : distribution gaussienne de moyenne nulle ; B : initialisée à zéro $\rightarrow \Delta W = BA = 0$ au démarrage
- La sortie est mise à l'échelle par α/r

$$h = W_0 x + (\alpha/r) \cdot B A x$$

Choix pratique : $\alpha = 16$ avec $r = 8$ (ratio 2) est courant dans PEFT/HuggingFace. L'essentiel : si vous changez r dans une ablation, gardez α constant pour mesurer l'effet de r seul.

Bilan paramétrique

Projection Q de CamemBERT-base ($d = k = 768$, $r = 8$)

Full fine-tuning

$768 \times 768 = 589\,824$ paramètres

LoRA $r=8$

$8 \times (768 + 768) = 12\,288$ paramètres

Ratio $\approx 48\times$. En adaptant Q et V sur les 12 couches : $12 \times 2 \times 12\,288 = 295\,000$ paramètres $\approx 0,27\%$ de 110M.

C'est ce ratio — environ 0,3 % — qui réduit l'empreinte mémoire de l'optimiseur d'un facteur 300.

Pourquoi Q et V, pas K

Rappel du Chapitre 1

W^Q — ce que l'on cherche

Adapter Q redirige l'attention vers des patterns pertinents pour la tâche (ex. "regarder" différemment autour des noms propres en NER médiévale).

W^V — ce qui est propagé

Adapter V change le vecteur qui alimente les couches supérieures quand un token est sélectionné.

W^K — rôle passif

Expose les tokens à la comparaison, mais le résultat est déjà capturé par les modifications de Q. Voita et al. (2019) : les clés sont les composantes les plus stables.

Ce choix n'est pas universel : pour la génération longue, adapter aussi W^O (target_modules incluant "o_proj") peut apporter un gain marginal. L'ablation tranchera pour votre tâche.

La fusion au moment de l'inférence

Un avantage pratique majeur de LoRA

$$W_{\text{merged}} = W_0 + (\alpha/r) \cdot B A$$

- Le modèle fusionné est structurellement identique à l'original : même nombre de paramètres, même architecture, latence inchangée
- Différence fondamentale avec les adapters, qui ajoutent des couches et augmentent la latence de façon permanente

```
model = AutoModelForSeq2SeqLM.from_pretrained("google/mt5-base")  
model = PeftModel.from_pretrained(model, "chemin/vers/lora_checkpoint")  
model = model.merge_and_unload() # modèle standard, sans overhead PEFT
```


PARTIE 3

Hyperparamètres LoRA : ce que chaque curseur fait

Le rang r

Contrôle la capacité de l'adaptation

- Plus r est grand, plus le sous-espace de mise à jour est riche, plus le modèle peut s'éloigner du pré-entraînement
- Compromis entre expressivité et régularisation implicite
- Valeurs courantes : $r = 4$ à 64 . Au-delà de 64 , on approche un full fine-tuning partiel
- Hu et al. (2022) : $r = 4$ ou 8 suffisent pour la plupart des tâches NLP — confirme l'hypothèse du faible rang intrinsèque

Hypothèse de travail pour la normalisation (~3000 paires) : $r=8$ devrait suffire, car les règles graphiques systématiques nécessaires sont peu nombreuses et régulières. Votre ablation $r=8$ vs $r=16$ le vérifiera.

α et le ratio α/r

α met à l'échelle la contribution de LoRA

$$\alpha/r = 1$$

Contribution LoRA de même ordre que les poids pré-entraînés — adaptation conservative

$$\alpha/r = 2$$

Ratio le plus courant dans les implémentations de référence

$$\alpha/r > 4$$

Adaptation agressive — risque de sur-apprentissage sur petits corpus

Recommandation pratique : fixer $\alpha = 2r$ pour les premières expériences, puis explorer si les résultats sont décevants.

Doubler r sans ajuster α doublerait la contribution de LoRA — le ratio évite que les comparaisons entre valeurs de r soient faussées.

Dropout LoRA et target_modules

Régularisation et choix des couches à adapter

- Dropout appliqué sur la sortie de A avant multiplication par B — force A et B à ne pas sur-spécialiser
- Corpus < 5000 exemples : dropout = 0.1 recommandé. Corpus plus larges : 0.05 ou 0

```
# CamemBERT / RoBERTa (encoder-only)
target_modules=["query", "value"]    # nommage RoBERTa

# mT5 / T5 (encoder-decoder)
target_modules=["q", "v"]            # nommage T5

# Lister les sous-modules disponibles :
{name for name, _ in model.named_modules() }
```

Récapitulatif des décisions d'hyperparamètres

Hyperparamètre	Valeur de départ	Quand l'ajuster
r	8	Augmenter si underfitting constaté ; ne pas dépasser 64
α	2r	Garder ce ratio constant dans les ablations
lora_dropout	0.1	Réduire si corpus > 10 000 exemples
target_modules	["q", "v"]	Ajouter "o" si tâche de génération longue
bias	"none"	"lora_only" si gain marginal attendu sur les biais

PARTIE 4

QLoRA : quantisation et LoRA combinés

Le problème de la quantisation naïve

float32 → int8 divise par 4 la mémoire... mais

- Les plages dynamiques des activations varient fortement selon les couches
- Un simple mapping linéaire vers $\{-128, \dots, 127\}$ introduit des erreurs de quantisation inacceptables sur les couches sensibles

QLoRA (Dettmers et al., 2023) introduit deux innovations qui rendent la quantisation 4 bits viable pour le fine-tuning : NF4 et la double quantisation.

NF4 (NormalFloat4)

Quantisation optimisée pour des poids gaussiens

$$q_i = Q_N(0,1)\left((i + 0.5) / 2^k \right), \quad i = 0, \dots, 2^k - 1 \quad (k = 4)$$

- Les poids d'un LLM pré-entraîné suivent approximativement une distribution gaussienne de moyenne nulle
- NF4 définit 16 niveaux de quantisation non uniformément espacés, positionnés sur les quantiles d'une gaussienne standard
- Erreur de quantisation $\sim 2\times$ inférieure à l'int4 uniforme

Double quantisation : les constantes de quantisation elles-mêmes sont quantisées en float8, réduisant leur empreinte de ~ 0.5 bit par paramètre.

Pagination mémoire avec bitsandbytes

Mémoire unifiée NVIDIA CPU / GPU

- Les pages mémoire CPU et GPU sont gérées comme un espace unifié par le driver NVIDIA
- Quand le GPU est saturé, les pages les moins utilisées sont déplacées vers la RAM CPU
- Permet d'entraîner des modèles qui dépasseraient sinon la capacité GPU, au prix d'une légère latence sur les transferts

Bilan mémoire : LLaMA-7B avec QLoRA

$r = 64, N_{LoRA} \approx 160M$

Composante	Float32	Float16	QLoRA (NF4 + LoRA)
Poids du modèle	4N octets	2N octets	~0.5N octets
Optimiseur (Adam)	8N octets	8N octets	~8N_LoRA octets
Activations	Variable	Variable	Variable

- Poids en NF4 : $0,5 \times 7 \times 10^9 \approx 3,5$ Go
- Optimiseur sur N_LoRA seulement : $8 \times 160 \times 10^6 \approx 1,3$ Go

Total estimé : < 6 Go — entraînable sur un T4 de 16 Go.

bitsandbytes en pratique, quand choisir QLoRA

Configuration NF4 + double quantisation

```
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.float16,
    bnb_4bit_use_double_quant=True,
)
model = AutoModelForSeq2SeqLM.from_pretrained(MODEL_NAME,
    quantization_config=bnb_config, device_map="auto")
model = prepare_model_for_kbit_training(model)
model = get_peft_model(model, lora_config)
# trainable params: 2,949,120 || all params: 302,432,256 (0.97%)
```

QLoRA n'est nécessaire que si le modèle ne tient pas en mémoire même en float16. Pour ce module (CamemBERT 110M, mT5-base 300M), LoRA standard suffit. QLoRA prend son sens pour mT5-large (1,2B), mT5-XL (3,7B), ou tout modèle > 1B sur un T4.

PARTIE 5

Les autres méthodes PEFT

Adapters (Houlsby et al., 2019)

Petits réseaux goulot d'étranglement entre les couches

$$\text{Adapter}(h) = h + W_{\text{up}} \cdot f(W_{\text{down}} \cdot h)$$

- Projection descendante vers $d_{\text{bottleneck}} \ll d_{\text{model}}$, activation non-linéaire, projection remontante, connexion résiduelle
- Avantage : multi-task learning — un modèle de base partagé + adapters séparés par tâche, swappés à l'inférence
- Inconvénient : latence d'inférence augmentée — les modules ajoutent des opérations sur le chemin critique

Pour ce module : moins pertinents que LoRA (pas de contrainte multi-tâche immédiate). Mais si au Jour 4 vous adaptez mT5 à la fois pour la normalisation et la coréférence, les adapters deviennent une option sérieuse.

Prefix Tuning et Prompt Tuning

Conditionner par des vecteurs entraînaables

Prefix Tuning (Li & Lam, 2021)

Préfixe les clés/valeurs de chaque couche d'attention avec $P_K, P_V \in \mathbb{R}^{(l \times d_{\text{model}})}$.
Compétitif sur la génération, mais plus difficile à optimiser et moins stable pour la classification / seq2seq courtes.

Prompt Tuning (Lester et al., 2021)

Préfixe uniquement la couche d'embedding d'entrée avec un vecteur de prompt p .
Simplicité extrême, mais faible expressivité — performant surtout sur les modèles > 10B paramètres.

Attention(Q, [P_K; K], [P_V; V]) — seuls les vecteurs du préfixe sont entraînés ; le modèle de base reste entièrement gelé.

(IA)³ — Infused Adapter

Liu et al., 2022 — la méthode PEFT la plus légère

- Apprend trois vecteurs de mise à l'échelle I_k , I_v , I_{ff} par couche
- Multipliés élément par élément avec les clés, les valeurs et les activations du FFN
- Environ 0,01% des paramètres entraînés — mais performances compétitives avec LoRA en few-shot

Intuition : inhiber et amplifier sélectivement certaines dimensions des activations suffit à reconfigurer le comportement du modèle pour une nouvelle tâche. Utile quand la mémoire est extrêmement contrainte ou pour minimiser l'overfitting sur un corpus minuscule.

PARTIE 6

Entraînement LoRA en pratique

recette pour mT5 et CamemBERT

mT5 pour la normalisation orthographique

Une tâche encoder-decoder : seq2seq avec préfixe de tâche

```
PREFIX = "normalise moyen français: "
```

```
# Exemple de paires brut -> normalisé
```

```
"au duc de norm~die" -> "au duc de normandie"
```

```
"li roys signa l acte" -> "le roi signa l acte"
```

```
lora_config = LoraConfig(  
    r=8, lora_alpha=16, target_modules=["q", "v"],  
    lora_dropout=0.1, bias="none",  
    task_type=TaskType.SEQ_2_SEQ_LM,  
)
```

learning_rate=3e-4 (plus élevé que pour full FT — LoRA converge vite), lr_scheduler_type="cosine", warmup_ratio=0.1, fp16=True, early stopping sur eval_loss.

CamemBERT pour la NER

Classification de tokens (labels BIO)

```
LABELS = ["O", "B-PER", "I-PER", "B-LOC", "I-LOC",  
          "B-DATE", "I-DATE", "B-ORG", "I-ORG", "B-TITLE", "I-TITLE"]
```

```
# Alignement labels <-> sous-tokens BPE :
```

```
# - premier sous-token d'un mot -> label original
```

```
# - sous-tokens suivants, [CLS], [SEP] -> -100 (ignorés)
```

```
lora_config = LoraConfig(  
    r=8, lora_alpha=16, target_modules=["query", "value"],  
    lora_dropout=0.1, bias="none",  
    task_type=TaskType.TOKEN_CLS,  
)
```

Le journal d'expériences

JSONL : un run, une ligne, tous les hyperparamètres

```
log_experiment(  
  config={"model": "google/mt5-base", "method": "lora",  
    "r": 8, "alpha": 16, "target_modules": ["q","v"],  
    "learning_rate": 3e-4, "epochs": 10, "batch_size": 8},  
  metrics={"eval_loss": 0.312, "token_accuracy": 0.847,  
    "cer_normalized": 0.043},  
  split_hash="10532c040c7639e3a05ff88a5b3773f5...",  
)
```

C'est l'artefact qui rend vos ablations interprétables : timestamp, hash du split, configuration complète, métriques de validation — tout est tracé dans `experiments/journal.jsonl`.

PARTIE 7

Scheduler cosine, warmup et arrêt prématuré

Le scheduler cosine

Le learning rate est le paramètre le plus sensible

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})(1 + \cos(\pi t / T))$$

- t = step courant, T = nombre total de steps
- Décroissance douce et continue de $\eta_{\max}$ à $\eta_{\min} \approx 0$
- Le modèle "explore" moins agressivement au fur et à mesure de la convergence

Un LR trop élevé en début d'entraînement détruit les représentations pré-entraînées avant convergence des matrices LoRA. Un LR trop faible en fin d'entraînement empêche l'exploration locale des dernières epochs.

Le warmup linéaire

warmup_ratio = 0.1 — un réglage robuste pour LoRA

- Démarrer avec un learning rate très bas (~ 0), puis l'augmenter linéairement jusqu'à $\eta_{\max}$ sur les premiers warmup_steps
- 10% des steps totaux est un réglage robuste pour LoRA

Intuition : au démarrage, A est aléatoire et B est nulle. Un grand LR immédiat causerait des mises à jour de grande amplitude qui perturbent le modèle avant que A et B ne soient "calibrées" sur la tâche. Le warmup laisse les matrices converger doucement vers un régime stable.

Arrêt prématuré (early stopping)

Une précaution indispensable pour les petits corpus

```
trainer = Seq2SeqTrainer(  
    model=model, args=training_args,  
    train_dataset=train_data, eval_dataset=val_data,  
    data_collator=data_collator,  
    callbacks=[EarlyStoppingCallback(  
        early_stopping_patience=3,      # 3 epochs sans amélioration  
        early_stopping_threshold=0.001, # amélioration min. significative  
    )],  
)
```

- Le modèle peut sur-apprendre les particularités du corpus après quelques epochs — continuer ne fait que dégrader la généralisation

Interaction avec cosine : si l'arrêt intervient à $t^ < T$, le LR peut être encore élevé. Certaines implémentations préfèrent alors `ReduceLROnPlateau`, qui s'adapte à la progression réelle de la validation.*

PARTIE 8

Connexion aux décisions du module

Le tableau d'ablation du Jour 2

Ce que chaque ligne teste

Configuration	CER val	Token acc.	Paramètres	Temps/epoch
Règles seules	—	—	0	< 1 s
LoRA r=8, Q+V	—	—	~295 K	—
LoRA r=16, Q+V	—	—	~590 K	—
LoRA r=8, Q+K+V+O	—	—	~590 K	—
Full FT (si mémoire dispo.)	—	—	300 M	—

r=8 vs r=16 : teste l'hypothèse du faible rang. Si r=16 fait significativement mieux, les mises à jour nécessaires dépassent la dimension 8 — tâche plus complexe ou cas ambigus non capturés par r=8.

Q+V vs Q+K+V+O : teste si l'adaptation des projections de sortie et des clés apporte un gain. Intuition : K est peu utile (Chapitre 1) ; W^O pourrait aider si la normalisation requiert des reformulations syntaxiques.

Interaction avec les scores de confiance HTR

Une exigence non négociable du cahier des charges

```
class WeightedSeq2SeqTrainer(Seq2SeqTrainer):
    def compute_loss(self, model, inputs, return_outputs=False, **kwargs):
        weights = inputs.pop("sample_weight", None)
        outputs = model(**inputs)

        if weights is not None:
            loss_fct = torch.nn.CrossEntropyLoss(reduction="none", ignore_index=-100)
            per_token_loss = loss_fct(logits.view(-1, logits.size(-1)), labels.view(-1))
            per_seq_loss = per_token_loss.view(labels.size()).mean(dim=-1)
            loss = (per_seq_loss * weights).mean()
        return (loss, outputs) if return_outputs else loss
```

Un pipeline qui ignore char_confidences pour pondérer la loss traite de la même façon une transcription fiable à 95% et une ligne bruitée à 65%. Le biais introduit sur le corpus d'entraînement se retrouvera dans les erreurs du modèle en production.

Ce qu'il faut retenir

- Le full fine-tuning est souvent inutile : les LLMs pré-entraînés ont déjà une représentation riche, et les mises à jour nécessaires ont un rang intrinsèque faible
- LoRA : $\Delta W = BA$, mise à l'échelle par α/r , fusion possible sans coût d'inférence — adaptez Q et V (pas K) sauf besoin spécifique de W^O
- $r=8$, $\alpha=16$, dropout=0.1 sont des points de départ robustes ; gardez α/r constant lors des ablations sur r
- QLoRA (NF4 + double quantisation + pagination mémoire) n'est nécessaire que pour les modèles > 1B sur GPU contraint
- Adapters, prefix/prompt tuning et (IA)³ répondent à d'autres contraintes (multi-tâche, mémoire extrême) — LoRA reste le choix par défaut de ce module
- Scheduler cosine + warmup + early stopping, et journal d'expériences JSONL avec hash de split : la reproductibilité est non négociable